

## Functions and Why use them

A **function** is simply a named block of code that performs a specific set of instructions.

### 1. The Anatomy of a Go Function

- **Declaration:** You define a function using the **func** keyword, followed by the **name**, parentheses **()**, and curly brackets **{}**.

```
func PrintHello() {  
    fmt.Println("Hello")  
}
```

- **Calling (Executing):** To run the code inside the function, you write its name followed by parentheses.

```
PrintHello()
```

- **The main Function:** This is the one exception to the rule. You never explicitly call **main()**. When you run a Go program, the computer automatically searches for **func main()** and executes it immediately. It is the starting line of your application.

### 2. The Power of Reusability

The most obvious reason to use functions is to avoid repeating yourself (**DRY principle: Don't Repeat Yourself**).

- Instead of writing the exact same 10 lines of code to connect to a database in 50 different places, you write it once inside a function called **ConnectToDB()**.
- Then, you simply call that function **50 times**. It drastically shrinks your codebase and makes it much easier to maintain.

### 3. The Power of Abstraction

Beyond saving keystrokes, functions provide **Abstraction**. Abstraction is the concept of hiding complex, messy details behind a simple interface so the human brain doesn't get overwhelmed.

- **How it works:** If you need to sort a massive list of user data, the actual mathematical algorithm to do that is incredibly complex. But if you wrap that algorithm in a function named **SortUsers()**, you no longer have to think about the math. You just call **SortUsers()** and trust that it works.
- **Self-Documenting Code:** When you use Abstraction properly, your code reads like a plain-English recipe.

```
func RegisterNewUser() {  
    ConnectDB()  
    ValidateUser()  
    CreateNewUser()  
}
```

Even without knowing Go, any developer can look at that **Register** function and instantly understand exactly what the program does.

**Expert Takeaway:**

Every single endpoint in your API (**GET /users** or **POST /login**) will eventually route to a specific Go function (often called a "**Handler**"). Abstraction is critical here!

Your Handler function should never contain raw SQL queries or complex data parsing. The Handler should simply call abstracted functions like **ValidateLogin()** and **FetchUserFromDB()**. This keeps your backend architecture modular, testable, and incredibly easy for other developers to read!